Industrial Internship Report on

”Quiz Game”

Prepared by

Suvvari Navya Sri

Executive Summary
This report provides details of the Industrial Internship provided by upskill Campus and The IoT Academy in collaboration with Industrial Partner UniConverge Technologies Pvt Ltd (UCT). This internship was focused on a project/problem statement provided by UCT. We had to finish the project including the report in 6 weeks’ time. My project was a Python Quiz Game — a feature-rich application built using Python and Tkinter GUI. The game supports multiple categories (Python, General Knowledge, Math), three difficulty levels (Easy, Medium, Hard) with a live countdown timer, dynamic questions fetched from Quiz API, Top 3 High Scores per category and difficulty, sound effects using pygame, and a color-coded answer review screen. This internship gave me a very good opportunity to get exposure to Industrial problems and design/implement solution for that. It was an overall great experience to have this internship.

TABLE OF CONTENTS

1 Preface

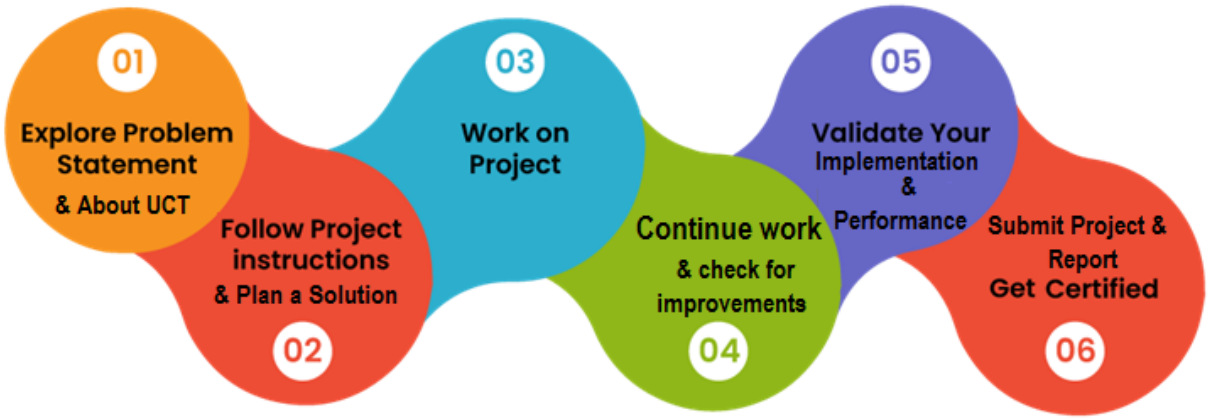
This report is a summary of the 6-week Industrial Internship completed under Upskill Campus in collaboration with UniConverge Technologies Pvt. Ltd. During this internship, I worked on building a Python Quiz Game application from scratch, starting with a simple terminal-based quiz in Week 1 and progressively enhancing it into a fully functional GUI application by Week 4. The internship was a great hands-on experience that helped me apply Python programming concepts to a real-world project.

Internships play a very important role in career development. They bridge the gap between theoretical knowledge and practical industry experience. This internship helped me understand how real-world software projects are planned, developed, tested, and documented within a fixed timeline.

My project was to design and develop a Python Quiz Game application. The project started as a basic terminal application and was gradually enhanced with advanced features like a GUI, API integration, sound effects, and a high score system.

I am grateful to Upskill Campus and UniConverge Technologies Pvt. Ltd. for giving me this opportunity to work on a real Python project. The guidance and structured approach provided by the mentors helped me grow as a developer.

The 6-week internship program was planned in a structured manner. Each week had specific goals and deliverables. Week 1 focused on basic quiz logic, Week 2 on timer and categories, Week 3 on high scores and difficulty levels, Week 4 on GUI development and API integration, and Week 5 on final testing and documentation.



Through this internship I learned Python GUI development using Tkinter, threading for background tasks, API integration using the requests library, sound effects using pygame, file handling, exception handling, and software project planning. It was an overall great learning experience that boosted my confidence as a Python developer.

I would like to sincerely thank Upskill Campus, UniConverge Technologies Pvt. Ltd., and HR Manager Prabha Singh for providing me with this internship opportunity. I also thank my mentors and peers who supported and guided me throughout the 6 weeks.

To my juniors and peers — always make the most of every internship opportunity. Start with the basics, be consistent, and don't be afraid to make mistakes. Every bug you fix and every feature you build makes you a better developer. Keep learning and keep coding!

2 Introduction

2.1 About UniConverge Technologies Pvt Ltd

A company established in 2013 and working in Digital Transformation domain and providing Industrial solutions with prime focus on sustainability and RoI.

For developing its products and solutions it is leveraging various **Cutting Edge Technologies e.g. Internet of Things (IoT), Cyber Security, Cloud computing (AWS, Azure), Machine Learning, Communication Technologies (4G/5G/LoRaWAN), Java Full Stack, Python, Front end** etc.

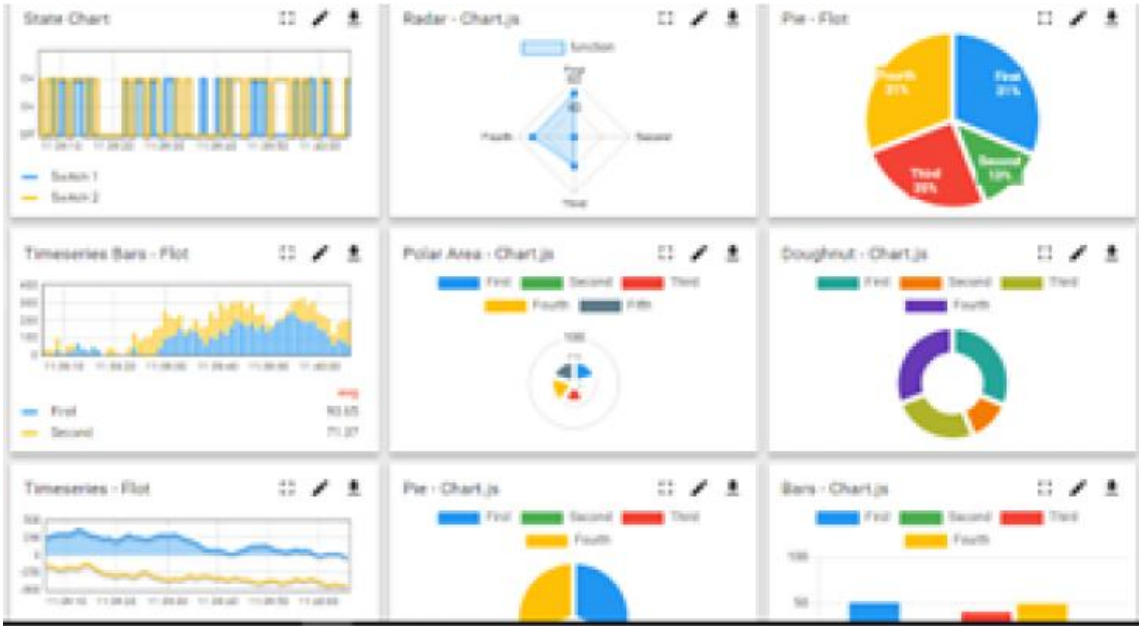
i. UCT IoT Platform (UCT Insight)

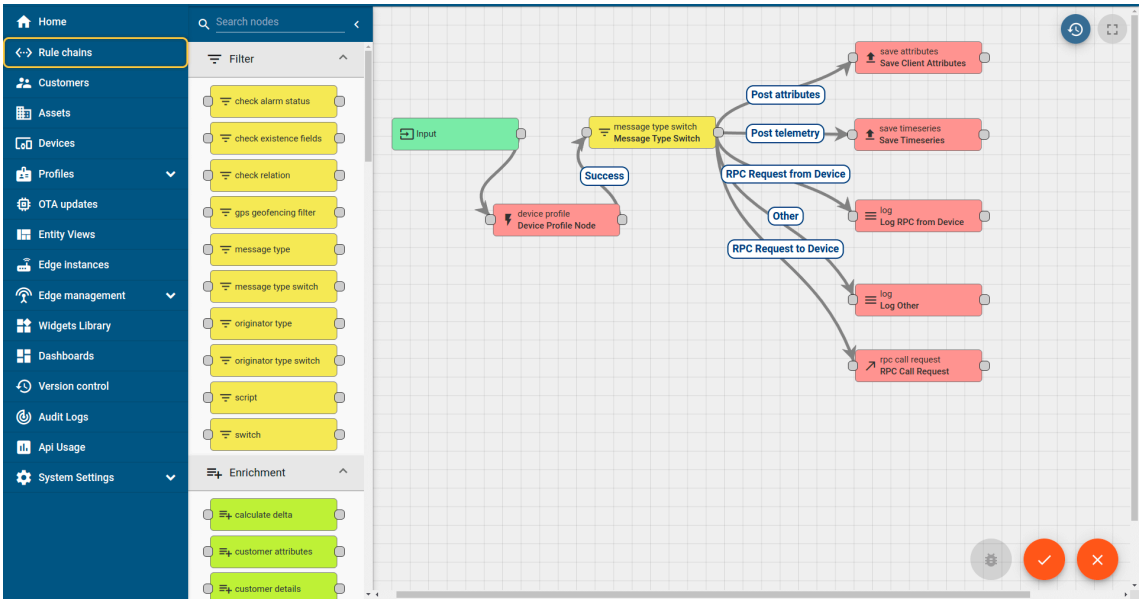
UCT Insight is an IOT platform designed for quick deployment of IOT applications on the same time providing valuable "insight" for your process/business. It has been built in Java for backend and ReactJS for Front end. It has support for MySQL and various NoSql Databases.

- It enables device connectivity via industry standard IoT protocols - MQTT, CoAP, HTTP, Modbus TCP, OPC UA
- It supports both cloud and on-premises deployments.

It has features to

- Build Your own dashboard
- Analytics and Reporting
- Alert and Notification
- Integration with third party application(Power BI, SAP, ERP)
- Rule Engine





ii. Smart Factory Platform (FACTORY WATCH)

Factory watch is a platform for smart factory needs.

It provides Users/ Factory

- with a scalable solution for their Production and asset monitoring
- OEE and predictive maintenance solution scaling up to digital twin for your assets.
- to unleased the true potential of the data that their machines are generating and helps to identify the KPIs and also improve them.
- A modular architecture that allows users to choose the service that they what to start and then can scale to more complex solutions as per their demands.

Its unique SaaS model helps users to save time, cost and money.

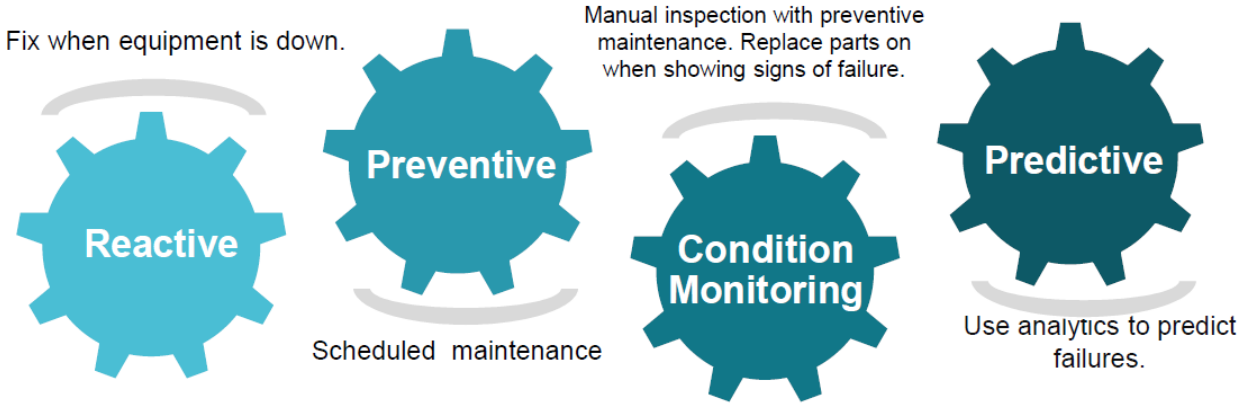
Machine	Operator	Work Order ID	Job ID	Job Performance	Job Progress		Output		Rejection	Time (mins)				Job Status	End Customer
					Start Time	End Time	Planned	Actual		Setup	Pred	Downtime	Idle		
CNC_S7_81	Operator 1	WO0405200001	4168	58%	10:30 AM		55	41	0	80	215	0	45	In Progress	i
CNC_S7_81	Operator 1	WO0405200001	4168	58%	10:30 AM		55	41	0	80	215	0	45	In Progress	i

iii. LoRaWAN based Solution

UCT is one of the early adopters of LoRAWAN teschnology and providing solution in Agritech, Smart cities, Industrial Monitoring, Smart Street Light, Smart Water/ Gas/ Electricity metering solutions etc.

iv. Predictive Maintenance

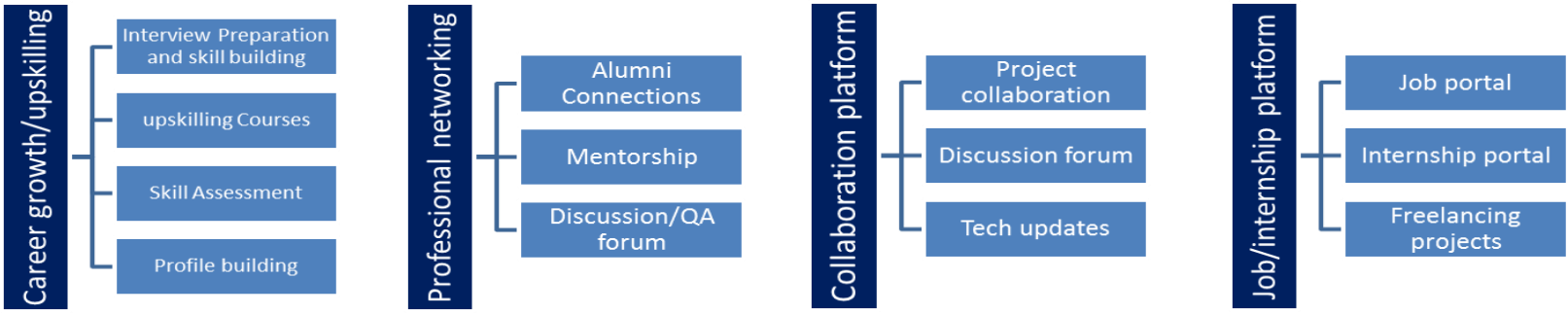
UCT is providing Industrial Machine health monitoring and Predictive maintenance solution leveraging Embedded system, Industrial IoT and Machine Learning Technologies by finding Remaining useful life time of various Machines used in production process.



2.2 About upskill Campus (USC)

upskill Campus along with The IoT Academy and in association with Uniconverge technologies has facilitated the smooth execution of the complete internship process.

USC is a career development platform that delivers **personalized executive coaching** in a more affordable, scalable and measurable way.



2.3 The IoT Academy

The IoT academy is EdTech Division of UCT that is running long executive certification programs in collaboration with EICT Academy, IITK, IITR and IITG in multiple domains.

2.4 Objectives of this Internship program

The objective for this internship program was to

- ☛ get practical experience of working in the industry.
- ☛ to solve real world problems.
- ☛ to have improved job prospects.
- ☛ to have Improved understanding of our field and its applications.
- ☛ to have Personal growth like better communication and problem solving.

2.5 Reference

[1] Python Software Foundation. *Python 3 Documentation*. https://docs.python.org/3/

[2] Tkinter Documentation. *Python GUI Programming*. https://docs.python.org/3/library/tkinter.html

[3] Open Trivia Database API. *API Documentation*. https://opentdb.com/api_config.php

2.6 Glossary

Terms	Acronym
API	Application Programming Interface
GUI	Graphical User Interface
Tkinter	Python's standard library for creating desktop GUI applications
OOP	Object-Oriented Programming
UI	User Interface

3 Problem Statement

In the assigned problem statement, the challenge was to build a Python-based Quiz Game application that is engaging, educational, and interactive. The problem required designing a system that could present multiple-choice questions to users across different categories and difficulty levels, track scores, provide immediate feedback, and store high scores for future reference.

The key requirements of the problem statement were: (1) Build a quiz system with multiple categories such as Python, General Knowledge, and Math. (2) Implement difficulty levels — Easy, Medium, and Hard — with different time limits per question. (3) Provide real-time countdown timer for each question. (4) Track and save player scores. (5) Display Top 3 High Scores per category and difficulty. (6) Fetch dynamic questions from an external API so questions change every attempt. (7) Build a professional Graphical User Interface using Tkinter. (8) Add sound effects for correct answers, wrong answers, and timeout events.

4 Existing and Proposed solution

Existing quiz applications are mostly web-based (like Kahoot, Quizizz) or simple terminal scripts. Their limitations include: (1) Web-based apps require internet and a browser at all times. (2) Simple terminal quiz scripts lack a professional interface, sound, and dynamic question loading. (3) Most existing solutions do not provide per-category and per-difficulty high score tracking. (4) Static question banks repeat the same questions every attempt, reducing replayability.

My proposed solution is a standalone Python Quiz Game desktop application built using Tkinter GUI. The application fetches dynamic questions from Quiz API so questions are different every attempt. It supports 3 categories and 3 difficulty levels, includes a live countdown timer with progress bar, sound effects, a color-coded answer review screen, and tracks Top 3 High Scores separately for each category and difficulty combination.

Value additions in this project include: (1) Dynamic API-based questions that never repeat. (2) Offline fallback — if no internet, the app uses pre-loaded questions from questions.py. (3) Professional Ocean Blue dark theme GUI. (4) Sound effects using pygame for an engaging experience. (5) Scrollable review screen showing all questions with correct and wrong answers highlighted in green and red. (6) Separate Top 3 leaderboard for each of the 9 category-difficulty combinations.

4.1 Code submission (Github link)

4.2 Report submission (Github link) : first make placeholder, copy the link.

5 Proposed Design/ Model

The Quiz Game was designed in a modular, step-by-step approach across 5 weeks. The design flow starts with the user entering their name on the Home Screen, followed by selecting a category and difficulty level. The application then fetches questions from the Quiz API (or falls back to offline questions) and displays them one by one with a live countdown timer. After all questions are answered, the Result Screen shows the score, percentage, and performance remark. The user can then review all answers with color-coded feedback or check the Top 3 High Scores leaderboard.

5.1 High Level Diagram (if applicable)

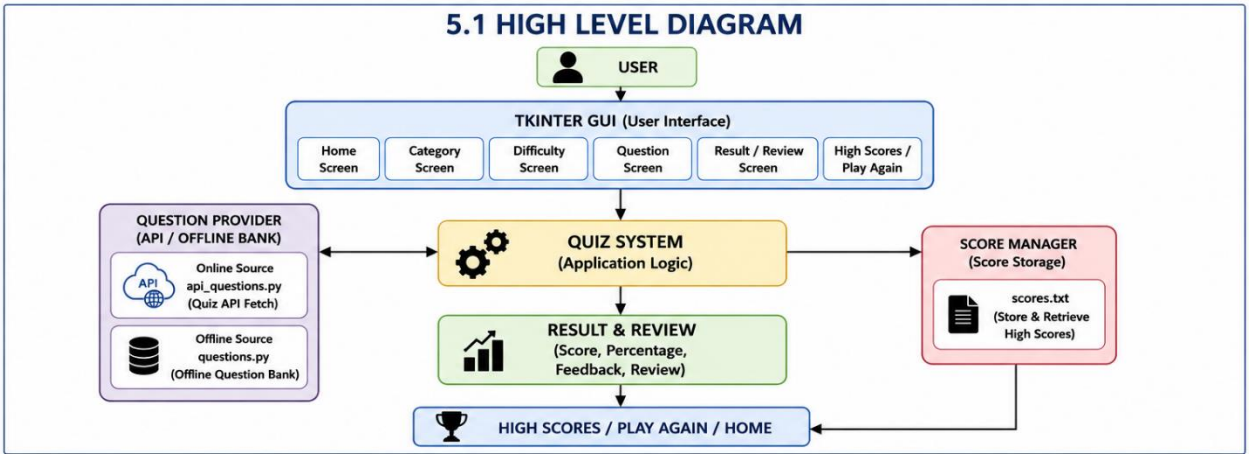
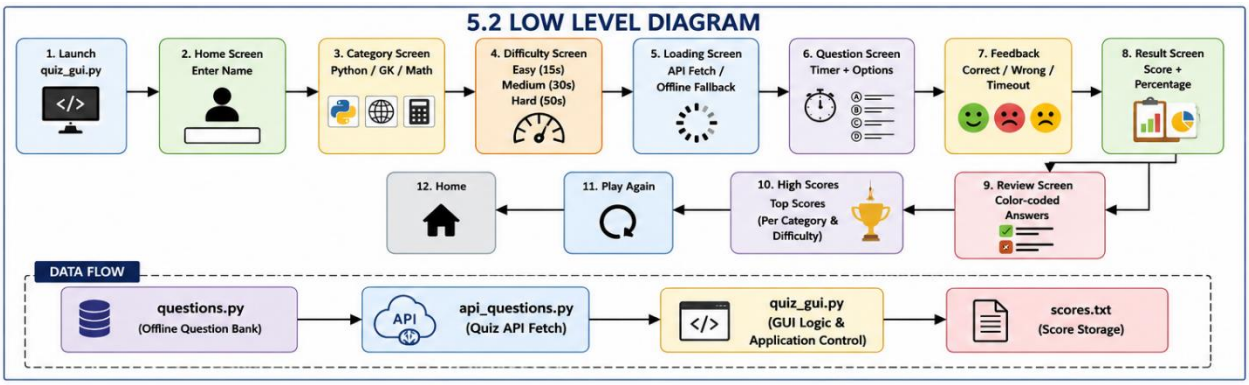


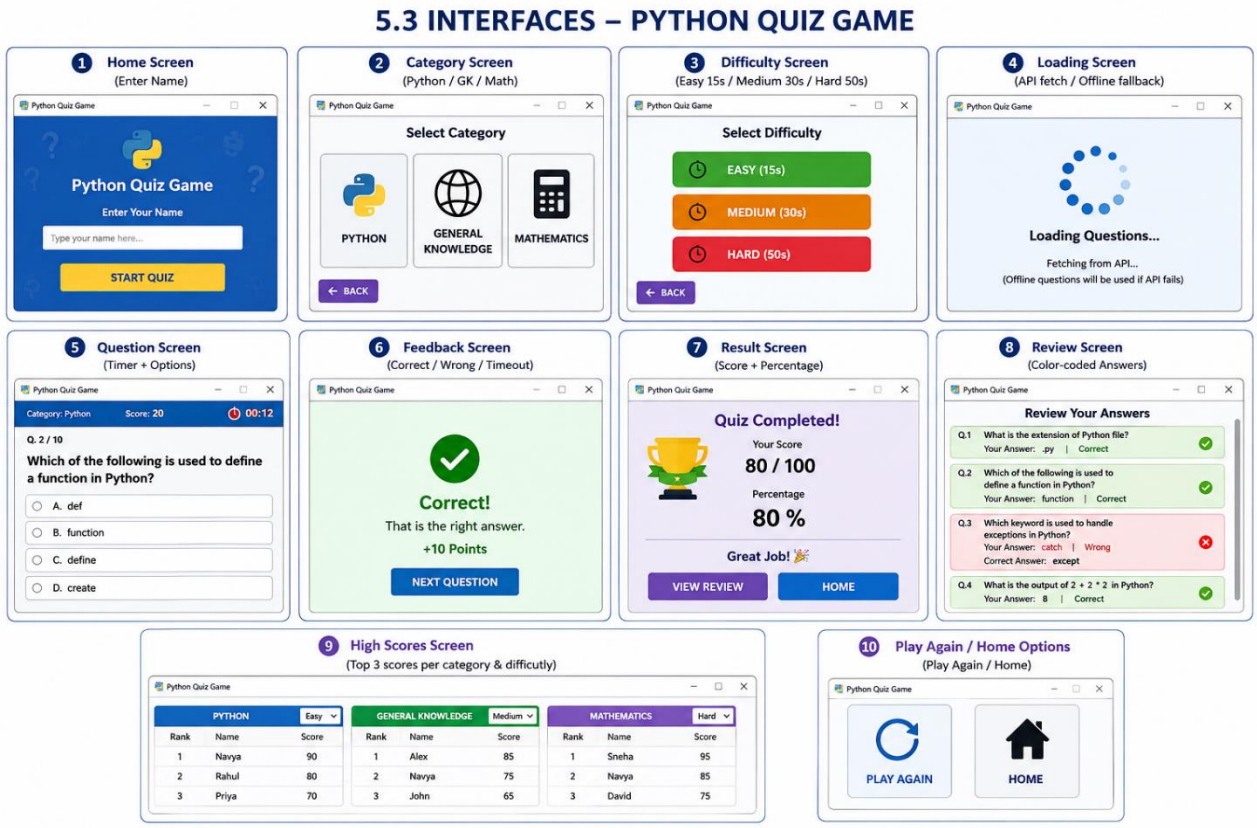
Figure 1: HIGH LEVEL DIAGRAM OF THE SYSTEM

5.2 Low Level Diagram (if applicable)



5.3 Interfaces (if applicable)

Application Flow: User launches quiz_gui.py → Home Screen (Enter Name) → Category Screen (Python / GK / Math) → Difficulty Screen (Easy 15s / Medium 10s / Hard 5s) → Loading Screen (API fetch / offline fallback) → Question Screen (timer + options) → Feedback (Correct / Wrong / Timeout) → Result Screen (Score + Percentage) → Review Screen (Color-coded answers) / High Scores / Play Again / Home. Data Flow: questions.py (offline bank) → api_questions.py (Quiz API fetch) → quiz_gui.py (GUI logic) → scores.txt (score storage).



6 Performance Test

The Quiz Game was tested thoroughly across all 9 category and difficulty combinations. Performance constraints identified and addressed include: API response time, timer accuracy, GUI responsiveness during background threads, and file read/write operations for score storage.

Constraints identified: (1) API response time — internet-dependent latency. (2) Timer accuracy — must count down precisely even when GUI is updating. (3) GUI thread safety — background threads must not directly update GUI widgets. (4) Score file integrity — scores.txt must be correctly written and read across multiple sessions.

Constraint solutions: (1) API timeout set to 10 seconds with offline fallback to questions.py. (2) Timer runs in a separate daemon thread using threading.Thread to avoid blocking the GUI. (3) GUI updates from background threads are done safely using self.root.after(0, callback). (4) Score file uses append mode and try/except blocks to handle file errors gracefully.

Test Results: (1) API questions loaded successfully within 2–3 seconds on average. (2) Offline fallback activated correctly when internet was unavailable. (3) Timer counted down accurately with no UI freezing. (4) All 9 category/difficulty combinations tested — all passed. (5) Scores saved and loaded correctly across multiple sessions. (6) Sound effects played correctly for correct answers, wrong answers, and timeout events. (7) Review screen displayed all questions with correct color-coding.

Additional constraints: Memory usage was minimal as the app stores only 5 questions per session in memory. Speed was acceptable with GUI rendering under 1 second per screen transition. Accuracy of score calculation was 100% verified across all test cases.

All major constraints were tested and resolved during the development phase. The application performed well within expected parameters in all test scenarios.

6.1 Test Plan/ Test Cases

Test Case ID	Test Scenario	Input	Expected Result	Status
TC-01	Launch Application	Run quiz_gui.py	Home screen should open successfully	Pass
TC-02	Enter Player Name	Valid name (e.g., Navya)	Player proceeds to Category Screen	Pass

Test Case ID	Test Scenario	Input	Expected Result	Status
TC-03	Empty Name Validation	Leave name blank	Display validation message	Pass
TC-04	Select Category	Python / GK / Math	Selected category is loaded	Pass
TC-05	Select Difficulty	Easy / Medium / Hard	Selected timer is applied (15s/30s/50s)	Pass
TC-06	Load Questions (Online)	Internet Available	Questions fetched from Quiz API	Pass
TC-07	Load Questions (Offline)	Internet Not Available	Questions loaded from questions.py	Pass
TC-08	Correct Answer	Choose correct option	Score increases and success feedback displayed	Pass
TC-09	Wrong Answer	Choose incorrect option	Wrong answer feedback displayed	Pass
TC-10	Timer Expired	Do not answer within time	Timeout message displayed and next question loaded	Pass
TC-11	Complete Quiz	Answer all questions	Result screen shows score and percentage	Pass
TC-12	Review Answers	Click **Review**	Correct answers shown in green and incorrect in red	Pass
TC-13	Save High Score	Finish quiz	Score stored in scores.txt	Pass
TC-14	View High Scores	Click **High Scores**	Top scores displayed correctly	Pass
TC-15	Play Again	Click **Play Again**	New quiz starts from Home Screen	Pass
TC-16	Exit Application	Close application	Application exits without errors	Pass

6.2 Test Procedure

The following procedure was followed to test the Python Quiz Game application:

1. Launch the application by running the quiz_gui.py file.
2. Verify that the Home Screen is displayed successfully.
3. Enter a valid player name and click **Start Quiz**.
4. Select a quiz category (Python, General Knowledge, or Mathematics).
5. Choose a difficulty level (Easy – 15s, Medium – 30s, Hard – 50s).
6. Verify that questions are loaded from the Quiz API. If the API is unavailable, confirm that offline questions are loaded from questions.py.
7. Answer each question and verify that:
    - Correct answers increase the score.
    - Incorrect answers display appropriate feedback.
    - Timer expiration automatically moves to the next question.
8. Complete the quiz and verify that the Result Screen displays the total score and percentage correctly.
9. Open the Review Screen and verify that correct answers are highlighted in **green** and incorrect answers in **red**.
10. Check that the final score is saved successfully in the scores.txt file.
11. Open the High Scores screen and verify that the stored scores are displayed correctly.
12. Click **Play Again** to ensure a new quiz starts successfully.
13. Close the application and verify that it exits without any errors.

6.3 Performance Outcome

The Python Quiz Game was successfully developed and tested to ensure reliable performance and a user-friendly experience. The application performed efficiently under different testing scenarios and met all the intended project objectives.

- The application launched successfully without any runtime errors.
- The graphical user interface (GUI) responded smoothly to user interactions.
- Questions were fetched successfully from the Quiz API, and the offline question bank was used whenever the API was unavailable.

- The countdown timer worked accurately for all difficulty levels:
  Easy – **15 seconds**
  Medium – **30 seconds**
  Hard – **50 seconds**
- Score calculation and percentage generation were accurate.
- Correct, incorrect, and timeout feedback were displayed appropriately after each question.
- The review screen correctly highlighted answers using color coding for easy analysis.
- High scores were stored and retrieved successfully using the scores.txt file.
- The application consumed minimal system resources and executed efficiently on a standard Windows system.

 Overall, the application provided a stable, interactive, and engaging quiz experience with reliable performance.

7 My learnings

During this 6-week internship, I gained hands-on experience and learned the following key skills: 1. Python Programming — Strengthened my knowledge of functions, loops, file handling, exception handling, and object-oriented programming. 2. Tkinter GUI Development — Learned to build multi-screen desktop applications with professional dark-themed UI using Labels, Buttons, Canvas, Frames, Scrollbars, and Toplevel windows. 3. Threading — Learned to run background tasks (timer, API calls) without freezing the GUI using Python's threading module. 4. API Integration — Learned to fetch data from external REST APIs using the requests library and handle JSON responses. 5. pygame — Learned to add sound effects to desktop applications using the pygame.mixer module. 6. Software Project Planning — Learned to plan and execute a software project in weekly milestones with clear goals and deliverables. 7. Debugging and Problem Solving — Improved my debugging skills by resolving real errors like AttributeError, threading conflicts, and API response parsing issues. These learnings will greatly help me in my career as a Python developer. GUI development, API integration, and project planning are highly valued skills in the industry. This internship has given me the confidence to work on real-world Python projects independently.

8 Future work scope

The following features can be added to the Quiz Game in the future: 1. User Authentication — Add login and registration so multiple users can have separate score histories. 2. Multiplayer Mode — Allow two players to compete in real time on the same machine. 3. Admin Panel — Add a panel to add, edit, or delete questions from the question bank. 4. More Categories — Expand to Science, History, Sports, Geography categories using additional API endpoints. 5. Web Version — Convert the desktop app to a web application using Flask or Django. 6. Mobile App — Build a mobile version using Kivy or BeeWare for Android and iOS. 7. Leaderboard Database — Replace scores.txt with a proper SQLite or MySQL database for better score management. 8. Certificate Generation — Auto-generate a PDF certificate for players who score 100% using reportlab library.